

LLMOrchestrator

LLMOrchestrator

(repo: `llm_orchestrator` · module `digital.vasic.llmorchestrator`)

Tagline: One control plane for every headless CLI coding agent.

Summary: LLMOrchestrator is a standalone, reusable Go module for spawning, managing, and communicating with headless CLI agents (OpenCode, Claude Code, Gemini, Junie, Qwen Code) over a hybrid pipe+file protocol, with per-agent circuit breakers, pluggable multi-provider selection, a decoupled i18n abstraction, and anti-bluff test guarantees.

Short description: A reusable Go module that gives one unified interface to spawn and drive multiple LLM-powered CLI agents via a hybrid pipe+file protocol. Thread-safe agent pooling with circuit breakers and selectable routing strategies, deliberately consumer-agnostic, with a pluggable i18n translator. (~40 words)

Long description: LLMOrchestrator is shared infrastructure for orchestrating headless CLI coding agents. Instead of every project re-implementing process spawning, message framing, and result parsing for tools like OpenCode, Claude Code, Gemini CLI, Junie, and Qwen Code, it provides one unified Agent interface, a thread-safe AgentPool, and a MultiProviderPool that manages agents from several providers behind one facade. Routing is pluggable via an AgentSelector — round-robin (skipping providers that don't meet requirements) or preference-ordered with fallback. Each concrete agent is a thin adapter over a shared BaseAdapter that owns the full process lifecycle (start with pipe setup, graceful SIGTERM-then-SIGKILL stop, restart, liveness). Communication is deliberately hybrid: a pipe transport carries newline-delimited JSON with a per-request read deadline and a response-length cap for fast interactive messaging, while a file transport uses per-session inbox/outbox/shared directories for large or durable artifacts. Resilience is built in: a per-agent circuit breaker opens after three consecutive failures for a 60-second cooldown before a half-open probe, and a background health monitor pings agents so they can recover without incoming traffic. Pool acquisition blocks on a condition variable rather than busy-waiting, and the response parser is stateless and safe to call concurrently. The module is

strictly decoupled — no consumer specifics leak in — and all user-facing strings pass through a pluggable i18n Translator, with a NoopTranslator that returns message IDs verbatim so missing translations surface visibly.

Why we built it: Every multi-agent system needs to launch and talk to CLI agents reliably. Re-solving spawning, framing, parsing, and failure-handling per project is wasteful and error-prone. LLMOrchestrator centralizes it into one decoupled, reusable module whose “specialised responsibility makes it reusable — and that reusability is destroyed the moment any consumer’s specifics leak in.”

Why it’s a game-changer: It turns “run an army of heterogeneous CLI agents” from a per-project engineering effort into a library concern — with pooling, circuit breaking, and pluggable routing already solved — and its anti-bluff tests prove the abstraction actually works end-to-end rather than merely compiling.

What’s innovative: - **Hybrid pipe+file protocol** — interactive speed (JSON-lines over stdin/stdout, read deadlines, response caps) plus durable file-based exchange (inbox/outbox/shared) for large artifacts. - **Multi-provider pool with pluggable selectors** — one facade over many CLI providers, round-robin or preference-ordered routing. - **Per-agent circuit breaker + background health monitor** — automatic degradation and recovery (3 failures → 60s open → half-open probe). - **Non-busy-wait pooling** — Acquire blocks on sync.Cond until a matching, healthy agent is free or the context is cancelled. - **Strict decoupling + anti-bluff i18n** — NoopTranslator surfaces missing translations rather than hiding them. - **Security-by-default** — binary-path allowlist (no shell interpolation → command-injection prevention), path-traversal protection, a 1 MiB response cap, and API-key masking in logs. - **Anti-bluff Challenge harness** — real disk/JSON/parser round-trips across five locales with a paired mutation gate that must exit non-zero when the feature is broken.

Biggest technical challenges + how solved: - **Reliable agent process I/O.** Solved with a hybrid pipe+file transport, a defined message/parser contract, and a BaseAdapter that centralizes process lifecycle (SIGTERM timeout → SIGKILL fallback). - **Concurrency without busy-waiting.** Solved with a mutex + condition-variable AgentPool (Acquire blocks until a capability-matched agent frees up) and a stateless, side-effect-free parser safe for concurrent use. - **Provider failure isolation.** Solved with per-agent circuit breakers plus a health-monitor goroutine that enables recovery without incoming requests. - **Proving correctness, not just compilation.** Solved with the round-275 Challenge runner: 29 invariants across en/sr/ja/es/de exercising the real system, plus a paired mutation gate (LLMORCH_MUTATE_RUNNER=1 must fail → wrapper exit 99) that proves

the gate itself isn't a bluff. - **Localization without silent failure.** Solved with the verbatim-id NoopTranslator seam and per-consumer translator injection.

Tech stack (why + how): - **Go (1.25)** — the module, its agent adapters, transports, and parser. - **Go stdlib only (+ testify, yaml.v3)** — deliberately minimal dependency surface; no LLM SDKs pulled in, keeping it embeddable. - **Pipe transport (JSON-lines over stdio)** — fast interactive messaging with read deadlines and response-length limits. - **File transport (inbox/outbox/shared)** — durable, large-artifact exchange per session. - **sync.Mutex/sync.Cond** — blocking, fair agent-pool acquisition. - **Circuit breaker + HealthMonitor** — per-agent resilience and recovery. - **pkg/i18n Translator** — decoupled localization seam. - **Challenge harness (challenges/runner) + Makefile (test -race, fuzz, cover)** — anti-bluff, evidence-backed verification including parser fuzzing.

Public links: - Product repo, PUBLIC: github.com/HelixDevelopment/LLMOrchestrator - Also referenced in docs as github.com/vasic-digital/LLMOrchestrator (clone URL in CONTRIBUTING/USER_GUIDE). - Governance reference: github.com/HelixDevelopment/HelixConstitution. - **Note:** the working origin remote is SSH (`git@github.com:HelixDevelopment/LLMOrchestrator.git`); the GitHub repo itself is public. License: Apache-2.0. Consumed as a submodule by multiple Helix/vasic projects.

Suggested diagrams/illustrations (OpenDesign): 1. **Control-plane fan-out** — LLMOrchestrator's MultiProviderPool spawning and driving OpenCode, Claude Code, Gemini, Junie, and Qwen Code, with a selector (round-robin / preference) choosing among them. 2. **Hybrid protocol** — side-by-side pipe path (interactive, JSON-lines, deadline+cap) vs file path (durable inbox/outbox/shared). 3. **Resilience loop** — per-agent circuit breaker state machine (closed → open → half-open) plus the health-monitor ping enabling recovery. 4. **Anti-bluff gate** — fixture → real parser/disk/JSON round-trip → asserted outcome, with the mutation branch shown failing (exit 99).

Site relevance: Both. Engineering-depth / AI-infrastructure feature on **vasic.digital**; reusable-module portfolio highlight on **milosvasic.ru**.

Priority tier: Helix-primary (LLM-infrastructure cluster — decoupled reusable module). Ranks after HelixTrack.

Source provenance: Local repo /Volumes/T7/Projects/llm_orchestrator — README.md, docs/ARCHITECTURE.md, root ARCHITECTURE.md, go.mod, .env.example, Makefile, pkg/agent/agent.go, CONTRIBUTING.md/USER_GUIDE.md, upstreams/setup-remotes.sh, CLAUDE.md/AGENTS.md/CONSTITUTION.md; cross-checked with harvested

_analysis/github-helix-others.md (deep-dive #14). Caution flags: CLAUDE.md §3.1 lists a much larger stack (Gin/PostgreSQL/Redis/Fyne/Bedrock) that describes the *parent helix_code app*, NOT this module — do not attribute. This module does NOT import LLMsVerifier/VisionEngine/DocProcessor directly; model metadata comes from LLMsVerifier bridged via HelixQA (ModelInfo/Score). No references to HelixTranslate or helix_cluster in this repo. “Helix-Flow” appears only as an org name in governance boilerplate.